

Generator drabinek turniejowych

– Sprawozdanie z testów

Bartosz Król, Nataniel Kwaśkiewicz - klasa 4D

Frontend

Testy napisane są w Jest (preset ts-jest) wraz z biblioteką React Testing Library (@testing-library/react). Każdy test renderuje komponent w symulowanym środowisku DOM dostarczonym przez jsdom, a następnie sprawdza widoczny wynik przy użyciu zapytań m.in. typu getByText, getByRole, getByTitle oraz asercji z @testing-library/jest-dom. Komponenty korzystające z React Query są owijane w QueryClientProvider, a hooki zewnętrzne zastępowane atrapami (jest.mock), dzięki czemu testy są w pełni izolowane od serwera i przeglądarki. Interakcje użytkownika symulowane są przez fireEvent, a testy zależne od czasu korzystają z fałszywych timerów (jest.useFakeTimers).

- 20 komponentów
- 157 testów

Arrow

Sprawdza, czy komponent renderuje element SVG oraz czy poprawnie stosuje domyślny rozmiar (24), niestandardowy rozmiar przekazany przez prop size oraz kolor obrysu przekazany przez prop color.

BracketParticipant

Weryfikuje wyświetlanie nazwy drużyny, renderowanie wyniku (w tym wartości 0) oraz brak elementu wyniku, gdy prop score nie jest podany.

BracketMatch

Testuje wyświetlanie nazw drużyn i wyników, otwieranie i zamykanie okna zmiany wyniku po kliknięciu meczu oraz poprawne przekazywanie zaktualizowanych wyników do callbacku onUpdateScore.

BracketRound

Sprawdza, czy wszystkie mecze rundy są renderowane, czy pojedynczy mecz wyświetla się prawidłowo oraz czy pusta lista meczów nie powoduje wyświetlenia żadnych elementów potomnych.

BracketGraph

Weryfikuje renderowanie nazw drużyn w jednej i wielu rundach oraz obecność elementu SVG służącego do rysowania linii łączących.

BracketSettings

Testuje domyślny stan rozwinięty, wstępne wypełnienie pola nazwy turnieju, zwijanie i rozwijanie panelu, blokowanie i odblokowywanie przycisku Save w zależności od zmian w polu nazwy, stany przycisku awansu (aktywny, turniej zakończony), a także przyciski przełączania tur dla formatu Round Robin wraz z wywołaniem callbacku onAdvanceLeg.

Bracket

Sprawdza, czy dla formatu eliminacyjnego wyświetlają się nazwy drużyn, a dla formatów Round Robin i Swiss pojawia się tabela Standings, oraz czy elementy charakterystyczne dla Round Robin nie są widoczne w trybie eliminacyjnym.

BracketContainer

Weryfikuje renderowanie nazwy turnieju i panelu ustawień, prawidłowe wyświetlanie widoków dla wszystkich trzech formatów (Single elimination, Round Robin, Swiss), stan końcowy turnieju eliminacyjnego, obecność przycisków zmiany rundy oraz brak błędów przy przełączaniu rund.

Carousel

Testuje renderowanie wszystkich slajdów z poprawnymi atrybutami src i alt, obsługę pojedynczego slajdu, automatyczne przechodzenie do kolejnego slajdu po 5 sekundach, zawijanie do pierwszego slajdu po ostatnim oraz poprawne sprzątanie interwału przy odmontowaniu komponentu.

ChangeScore

Sprawdza nagłówek okna, etykiety z nazwami drużyn, wstępne wypełnienie pól wynikami z propsów (w tym domyślne 0), wywołanie callbacków onCancel i onConfirm z

poprawnymi wartościami po interakcji użytkownika oraz ograniczenie wartości wejściowych do zakresu 0–9999.

Footer

Weryfikuje obecność nazwy aplikacji, aktualnego roku, imion autorów oraz elementu `<footer>`.

FormatGrid

Testuje wyświetlanie kart z nazwami i opisami wszystkich formatów, renderowanie obrazka tylko dla formatów posiadających `image_path`, a także zachowanie przy pustej liście i wartości `undefined`.

NavBar

Sprawdza obecność logo z linkiem do strony głównej, wszystkich trzech linków nawigacyjnych wraz z poprawnymi wartościami `href` oraz elementu `<nav>`.

RoundRobinTable

Weryfikuje nagłówek Standings, nazwy drużyn, wyniki rozegranych meczów (wygrana i remis), wskaźnik oczekującego meczu, kolumny tabeli wynikowej, otwieranie i zamykanie okna zmiany wyniku oraz renderowanie komórek przekątnej dla spotkań tej samej drużyny ze sobą.

SwissTable

Testuje nagłówek Standings, nazwy drużyn, zakładki rund, licznik rozegranych rund, domyślne wyświetlanie najnowszej rundy, przełączanie zakładek, wyświetlanie wyników (wygrana, remis, nierozegrane), otwieranie i zamykanie modalu zmiany wyniku, kolumny tabeli wynikowej oraz zachowanie przy braku meczów.

TeamForm

Sprawdza tytuł formularza, pole nazwy drużyny, przycisk tworzenia, domyślną liczbę członków, kłamrowanie liczby członków (min. 1, maks. 32), inkrementację i dekrementację licznika, blokowanie wysyłki przy pustej nazwie, poprawne przekazanie danych (z przycinaniem spacji) do mutacji, listę rozwijalną krajów oraz zachowanie, gdy pola członków są puste.

TeamHeader

Weryfikuje wyświetlanie nazwy drużyny, zastosowanie klasy CSS i atrybutu title, brak tooltipa przed najechaniem myszą oraz pojawianie się i znikanie TeamTooltip odpowiednio przy zdarzeniach mouseenter i mouseleave.

TeamTooltip

Testuje stan ładowania, wyświetlanie nazwy drużyny i imion członków, renderowanie pseudonimu w cudzysłowie, brak elementu pseudonimu, gdy jest null, flagę kraju z poprawnym src, komunikat „No members” dla pustej listy, pozycjonowanie tooltipa na podstawie propsa position oraz pusty widok, gdy dane nie są dostępne.

TournamentCard

Sprawdza wyświetlanie nazwy turnieju i formatu, liczby drużyn (z poprawną odmianą „team”/„teams”), nawigację do podstrony turnieju po kliknięciu Otwórz oraz wywołanie mutacji usuwania z poprawnym ID po kliknięciu przycisku usuń.

TournamentForm

Weryfikuje nagłówek i pola formularza, przyciski formatów i drużyn, stany ładowania i błędu dla danych zewnętrznych, walidację (brak nazwy, niepoprawna liczba drużyn dla eliminacji), odpowiedź dozwolonych rozmiarów, przełączanie wyboru drużyn, poprawny ładunek przekazywany do mutacji, stan oczekiwania podczas tworzenia turnieju oraz dezaktywację przycisku wysyłki.

Backend

Testy backendowe także napisane są w **Jest**, który obsługuje TypeScript bezpośrednio bez osobnego kroku kompilacji. Każdy test wywołuje funkcję kontrolera Express bezpośrednio (bez uruchamiania serwera HTTP), przekazując obiekty req, res i next z pomocniczego pliku testHelpers.ts. Klient Prisma jest pobierany z biblioteki jest-mock-extended (mockDeep<PrismaClient>()). Dzięki temu żaden test nie działa na prawdziwej bazie danych. Testy sprawdzają zarówno kody odpowiedzi HTTP, jak i konkretne dane przekazywane do metod Prisma.

- 2 controllery
- 8 funkcji
- 44 testy

Testy dla MatchesController

generateMatches

Sprawdza walidację wejściową - zwracanie statusu 400, gdy w turnieju są mniej niż 2 drużyny oraz gdy ich liczba jest nieparzysta (format eliminacyjny wymaga parzystej). Weryfikuje też poprawną generację meczów: dla n drużyn powinno powstać dokładnie $n/2$ par w rundzie 1 z wynikami 0:0, przy czym każda drużyna pojawia się dokładnie raz. Na końcu sprawdza, że błąd Prisma jest przekazywany do `next(error)`.

generateRoundRobinMatches

Testuje, czy próba wygenerowania meczów dla turnieju, który już je ma, kończy się kodem 400 bez wywołania `createMany`. Sprawdza też odrzucenie żądania przy mniej niż 2 drużynach. Dla poprawnych danych weryfikuje liczbę generowanych meczów - przy n drużynach ma powstać dokładnie $n*(n-1)$ rekordów (dwie rundy, każda para gra raz w każdą stronę), zarówno dla parzystej, jak i nieparzystej liczby drużyn. Potwierdza też odpowiedź 201 z listą meczów oraz obsługę błędu Prisma.

generateSwissMatches

Analogicznie do Round Robin sprawdza blokadę podwójnej generacji i walidację minimalnej liczby drużyn. Główna część testów koncentruje się na obsłudze nieparzystej liczby uczestników: przy parzystej liczbie drużyn powstaje $n/2$ prawdziwych meczów, przy nieparzystej — $\text{floor}(n/2)$ meczów plus dokładnie jeden rekord `bye`, gdzie `teamAId === teamBId`, oznaczony jako `played: true` z wynikiem 1:0 (automatyczna wygrana). Sprawdza też odpowiedź 201 i propagację błędu Prisma.

advanceSwissRound

Testuje wszystkie ścieżki blokujące awans: brak meczów w danej rundzie (404), nierozegrane prawdziwe mecze (422) oraz osiągnięcie limitu rund obliczonego jako $\text{ceil}(\log_2(n))$ dla danej liczby drużyn (422 z komunikatem zawierającym „complete”). Weryfikuje, że mecz `bye` nie jest traktowany jako nierozegrany i nie blokuje przejścia do kolejnej rundy. Sprawdza też, że dla parzystej liczby drużyn powstają poprawne $n/2$ par, a przy nieparzystej, drużyna bez pary dostaje nowy mecz `bye` z flagą `played: true`. Na koniec potwierdza odpowiedź 201 z meczami nowej rundy i obsługę błędu Prisma.

advanceRound

Sprawdza dwa przypadki blokujące: gdy w rundzie pozostał tylko jeden mecz (turniej już zakończony - 422), oraz gdy którykolwiek mecz ma remis i nie da się wyłonić zwycięzcy (422). Weryfikuje poprawne wyłanianie zwycięzców - zarówno gdy wygrywa teamA, jak i gdy wyższy wynik ma teamB - i że trafiają oni jako pary do następnej rundy z inkrementowanym numerem rundy. Potwierdza odpowiedź 201 z meczami kolejnej rundy i przekazanie błędu Prisma do next.

updateMatchScore

Testuje zwrócenie 404, gdy mecz o podanym ID nie istnieje. Dla istniejącego meczu sprawdza, czy prisma.match.update jest wywoływane z poprawnymi wynikami i flagą played: true, a wynik 0:0 również oznacza mecz jako rozegrany. Weryfikuje odpowiedź 200 z zaktualizowanym obiektem meczu oraz obsługę błędu Prisma.

Testy dla TournamentController

postTournament

Sprawdza, czy turniej jest tworzony z poprawnymi danymi - nazwą, formatem oraz listą drużyn przekazaną jako TournamentTeam.create - i że funkcja zwraca 201 wraz z nowo utworzonym obiektem. Weryfikuje też, że data przekazana jako string jest zamieniana na obiekt Date przed zapisem do bazy. Na koniec testuje przekazanie błędu Prisma do next(error).

updateTournament

Testuje walidację - brak nazwy w ciele żądania powinien zwrócić 400 z komunikatem „Tournament name is required” bez wywołania prisma.tournament.update. Sprawdza, że nazwa z nadmiarowymi spacjami jest przycinana przed zapisem. Weryfikuje poprawną odpowiedź 200 z zaktualizowanym turniejem oraz obsługę błędu Prisma.

```
Test Suites: 22 passed, 22 total
Tests:       201 passed, 201 total
Snapshots:   0 total
Time:        4.69 s
Ran all test suites.
```